

Stage 1

Description and Goals

Stage 1 involves creating the player with adequate movement abilities. This extends to attacks, which for now just involves changing the sprite.

To Do List

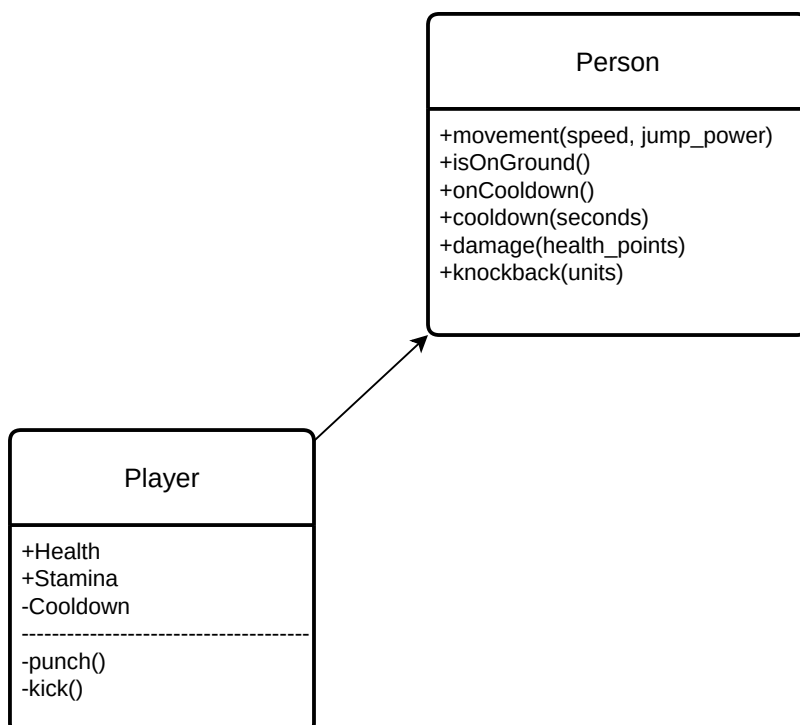
Features

- ✓ Player
 - ✓ Can move with keyboard input (left, right, jump)
 - ✓ Can attack with keyboard input
 - ✓ Player returns to ground after jumping

Tests

- ✓ Player sprite displayed?
- ✓ Accurate movement?
- ✓ Attack animation displayed?
- ✓ Gravity?
- ✓ Health Values?

Class Diagram

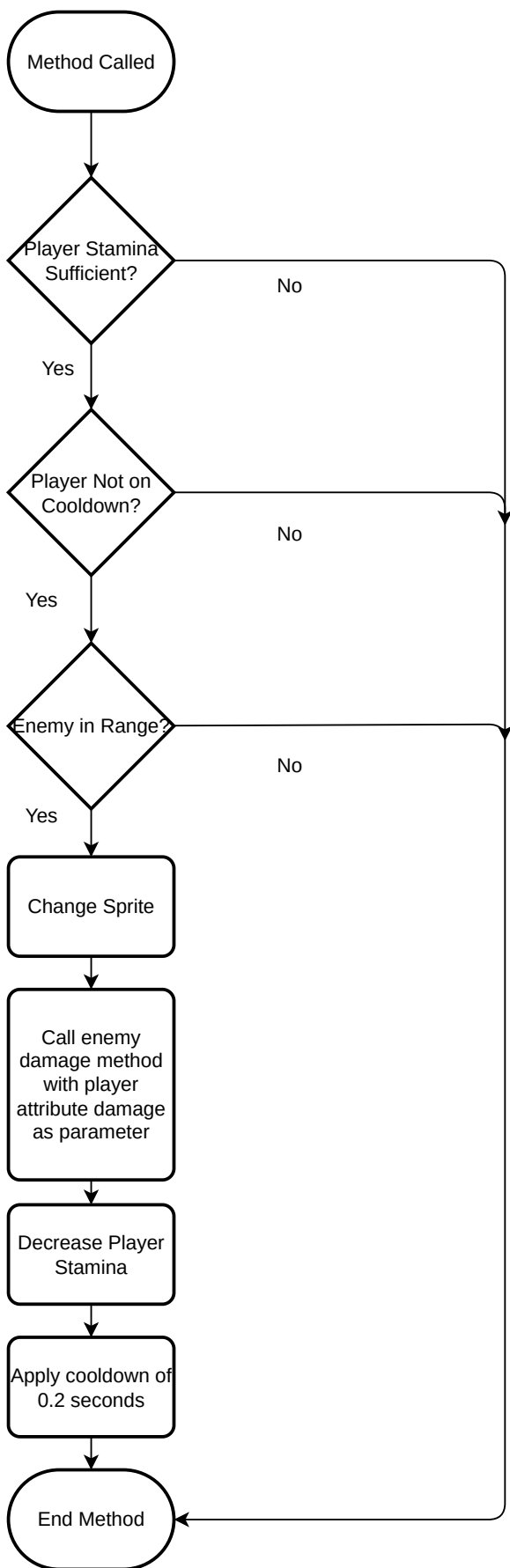


This class diagram holds all of the attributes and methods for my player. I have produced this because it

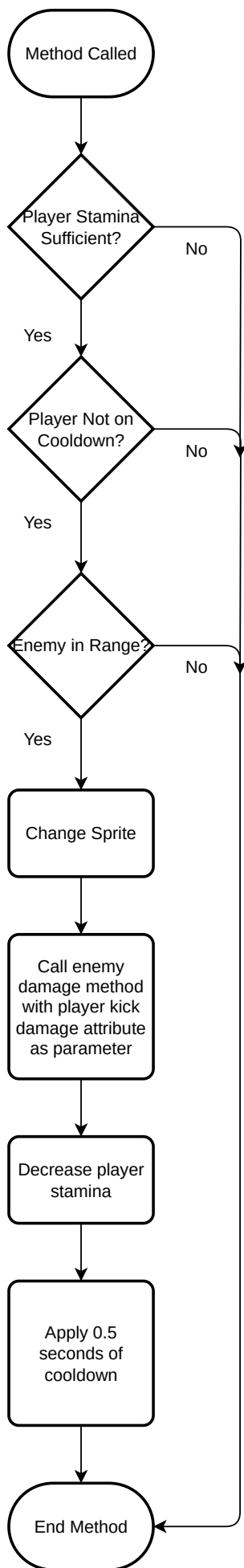
will be incredibly useful to know all of the methods which I will need to implement and attributes to declare. Please note that this would not include all variables. Ideally, I would have some declared like constants to make it easier to modify data such as horizontal acceleration.

Methods

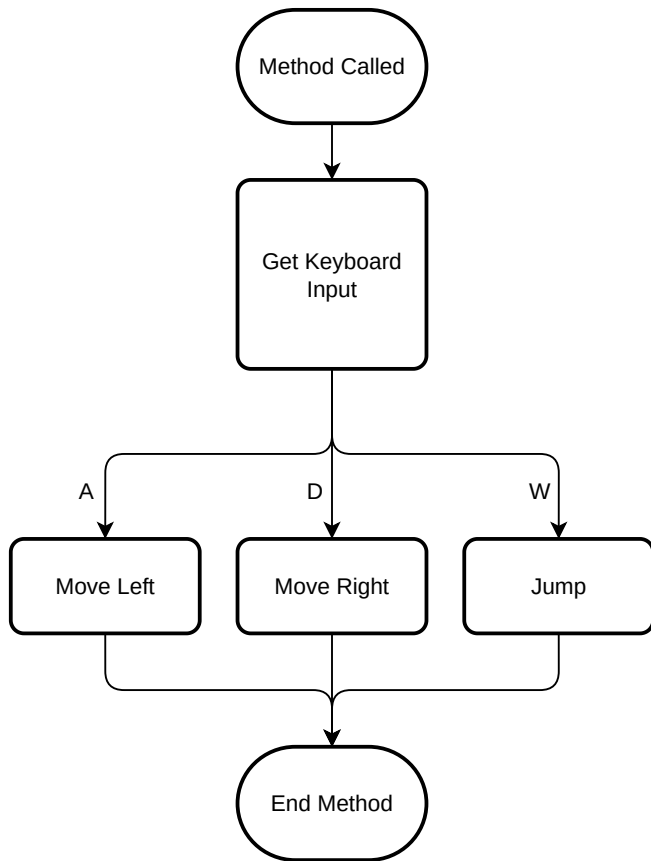
punch()



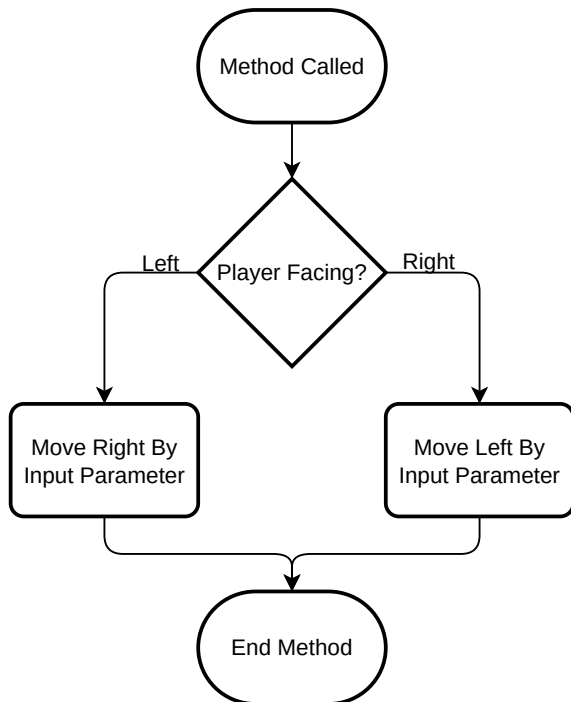
kick()



movement()



knockback()



damage()

```
function damage(int hp) then
    this.health = this.health - hp;
end function
```

Attributes

Health

Attribute to control the character's health. When `player.health == 0`, game over.

Stamina

Controls whether or not the player is allowed to attack.

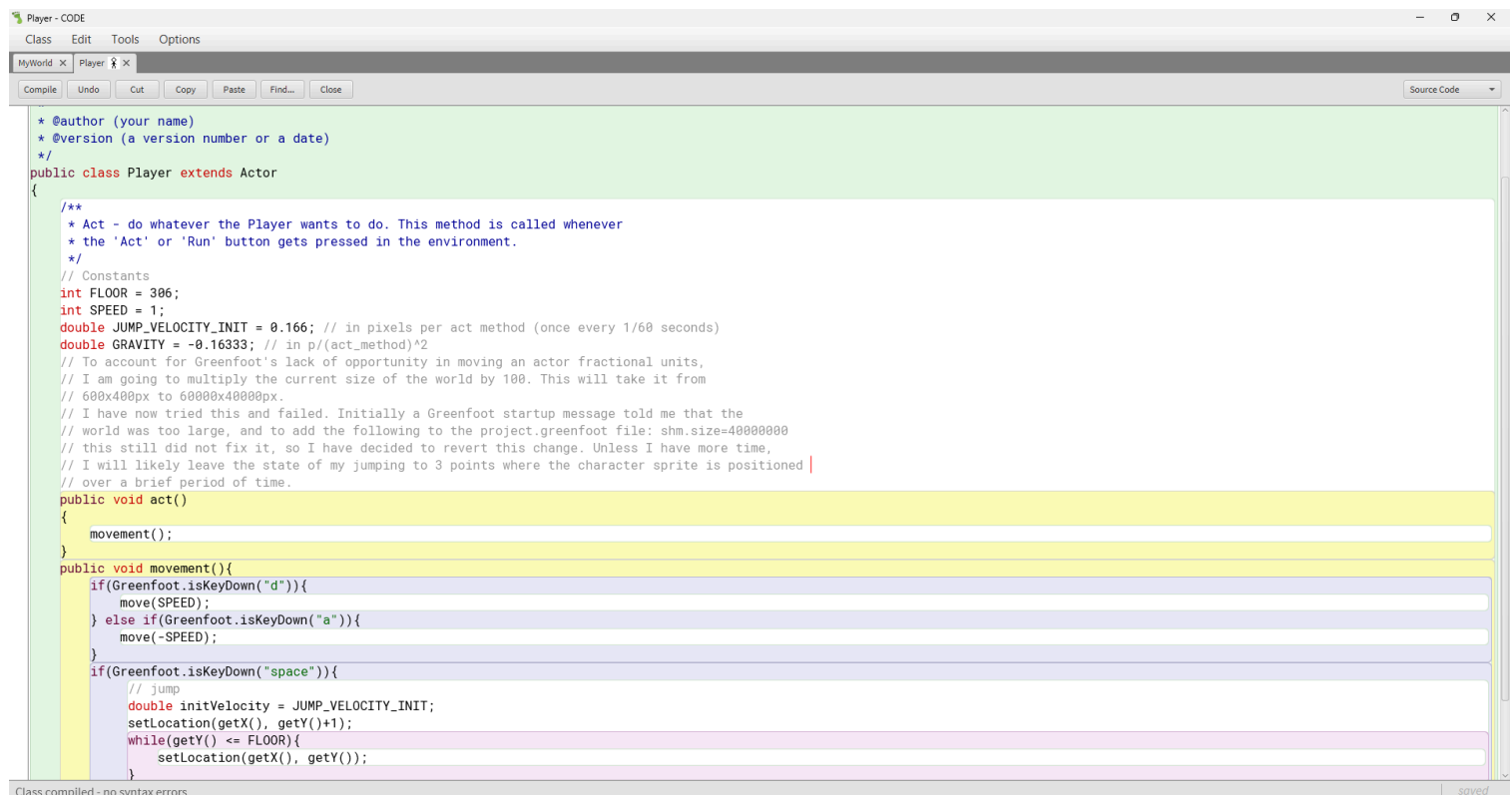
Cooldown

Same as `stamina` but with more direct consequences. Stamina is designed to limit and restrict against attacking relentlessly, whilst `cooldown` should stop the player from being able to attack every frame (too fast).

Notes

[references > How to round to an integer](#)

Pixel Rounding Error

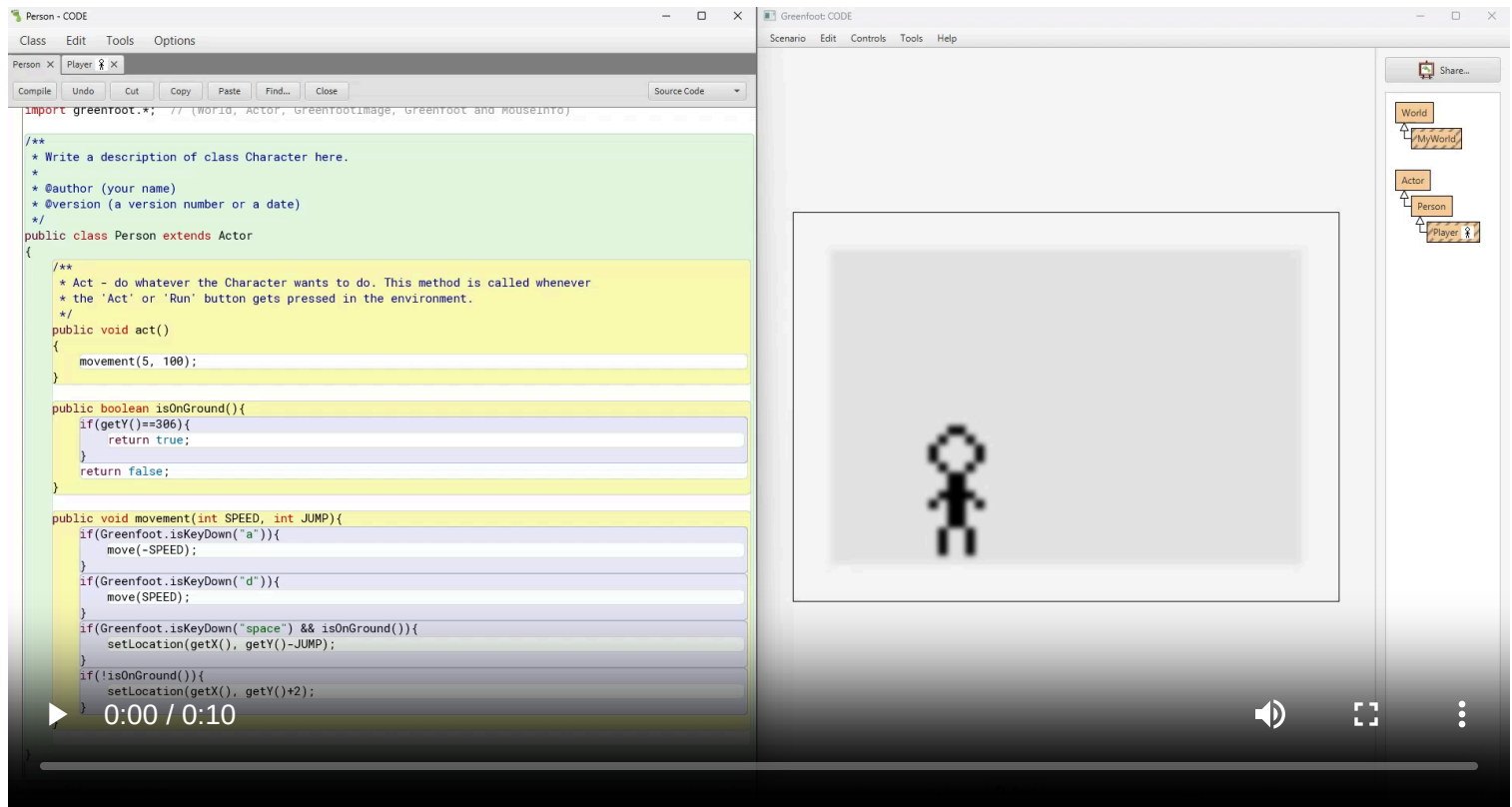


```
Player - CODE
Class Edit Tools Options
MyWorld x Player x
Compile Undo Cut Copy Paste Find... Close Source Code
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class Player extends Actor
{
    /**
     * Act - do whatever the Player wants to do. This method is called whenever
     * the 'Act' or 'Run' button gets pressed in the environment.
     */
    // Constants
    int FLOOR = 306;
    int SPEED = 1;
    double JUMP_VELOCITY_INIT = 0.166; // in pixels per act method (once every 1/60 seconds)
    double GRAVITY = -0.16333; // in p/(act_method)^2
    // To account for Greenfoot's lack of opportunity in moving an actor fractional units,
    // I am going to multiply the current size of the world by 100. This will take it from
    // 600x400px to 60000x40000px.
    // I have now tried this and failed. Initially a Greenfoot startup message told me that the
    // world was too large, and to add the following to the project.greenfoot file: shm.size=40000000
    // this still did not fix it, so I have decided to revert this change. Unless I have more time,
    // I will likely leave the state of my jumping to 3 points where the character sprite is positioned |
    // over a brief period of time.
    public void act()
    {
        movement();
    }
    public void movement(){
        if(Greenfoot.isKeyDown("d")){
            move(SPEED);
        } else if(Greenfoot.isKeyDown("a")){
            move(-SPEED);
        }
        if(Greenfoot.isKeyDown("space")){
            // jump
            double initVelocity = JUMP_VELOCITY_INIT;
            setLocation(getX(), getY()+1);
            while(getY() <= FLOOR){
                setLocation(getX(), getY());
            }
        }
    }
}
```

This is the first major error which I encountered. I was overambitious in my designing of a jump method, and wanted it to be physically accurate, using equations of motion. The problem with this was that Greenfoot would not have been able to move an object a fractional number of pixels, meaning that my

JUMP_VELOCITY_INIT and GRAVITY variables would not be viable, even if I were to round them. I tried to solve this by multiplying the number of pixels in the world by 100, in order to do the same to the variables and reduce the rounding errors. This would not work due to Greenfoot's own limit on pixels. This led to me having to scrap this idea.

Moving and Jumping Proof



In this video (moving_and_jumping.mp4), we can see the player moving and jumping, through the inherited move method from the parent class `Person`. The jumping is quite janky because of the use of `setLocation(getX(), getY()-JUMP)`. I tried to incorporate a gradual increase such as

```
while(getY() >= FLOOR - JUMP){
    setLocation(getX(), getY() - 5);
}
while(getY() < FLOOR){
    setLocation(getX(), getY() + 5);
}
```

But this became a problem with Greenfoot. I am still unsure why, but it would cause the program to crash. The current solution is now a compromise.

Old Cooldown Method

```

56 // setters
57 public void cooldown(double t){ // t in seconds
58     this.cooldown = t*60; // greenfoot tick speed is 60 per second, this allows t to be passed as seconds
59     for(int c = this.cooldown; c>=0; c--){ // uses this.cooldown to reference object's value
60         this.cooldown--; // eventually sets cooldown back to 0
61     }
62 }
63
64

```

■ Type mismatch: cannot co

This method was the old `cooldown` method before the new one

(

```

import java.math.RoundingMode;
import java.text.DecimalFormat;

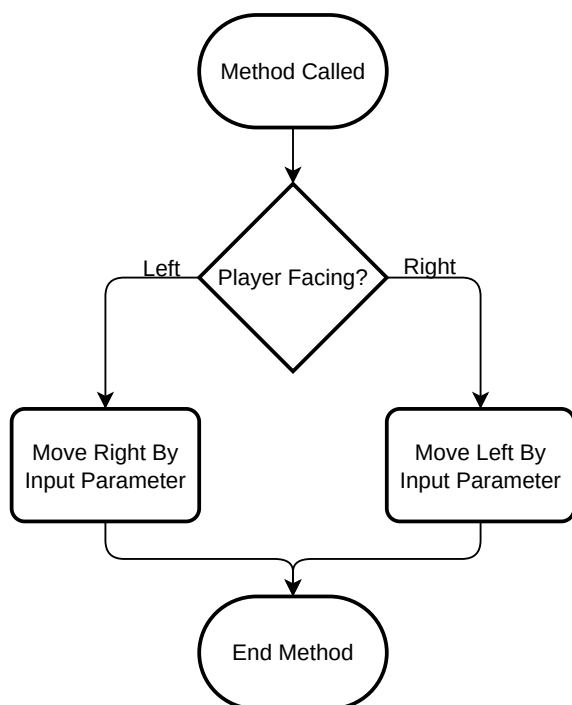
public void cooldown(double t){
    double unrounded_cooldown = t*60;
    DecimalFormat df = new DecimalFormat("###"); // where unused digits are 0
    df.setRoundingMode(RoundingMode.CEILING); // rounds up from t*60
    this.cooldown = df.format(unrounded_cooldown); // sets the cooldown to
appropriate integer
    for(int c = this.cooldown; c>=0; c--){ // same logic as if it were an int
        this.cooldown--;
    }
}

```

). As can be seen, `this.cooldown` is incorrectly assigned as `t*60`. This is because I had forgotten my initial plan to have the punch cooldown as 0.2 seconds. This adjustment was the reason which I had to change to an algorithm with explicit rounding.

With regards to Knockback

I have at present chosen not to implement the full `Knockback` method as was planned in my flowchart.



This is because I think that finding out which way the player was facing (especially in Greenfoot), would

take quite a lot of time. Therefore, I have at present left this either for the future or permanence. The current method stands at:

```
public void knockback(int k){  
    move(-k);  
}
```

This takes a heuristic approach, assuming that the player will always be on the left, attacking right. It is far more concise than the flow chart demands it be, which is why I have decided that it is currently the best option.

Fixing A Rounding Error

In the previous fix for the `cooldown` method, I employed the libraries `math.RoundingMode` and `text.DecimalFormat`. This is shown in [Old Cooldown Method](#). I have since dealt with an error produced by the `df.format()` call. As it happens, this method returns a string type, which caused a problem with `cooldown`, being defined as an integer. I fixed this by adding

```
Integer.parseInt(df.format(target));
```

thus converting the "string" to the desired integer.

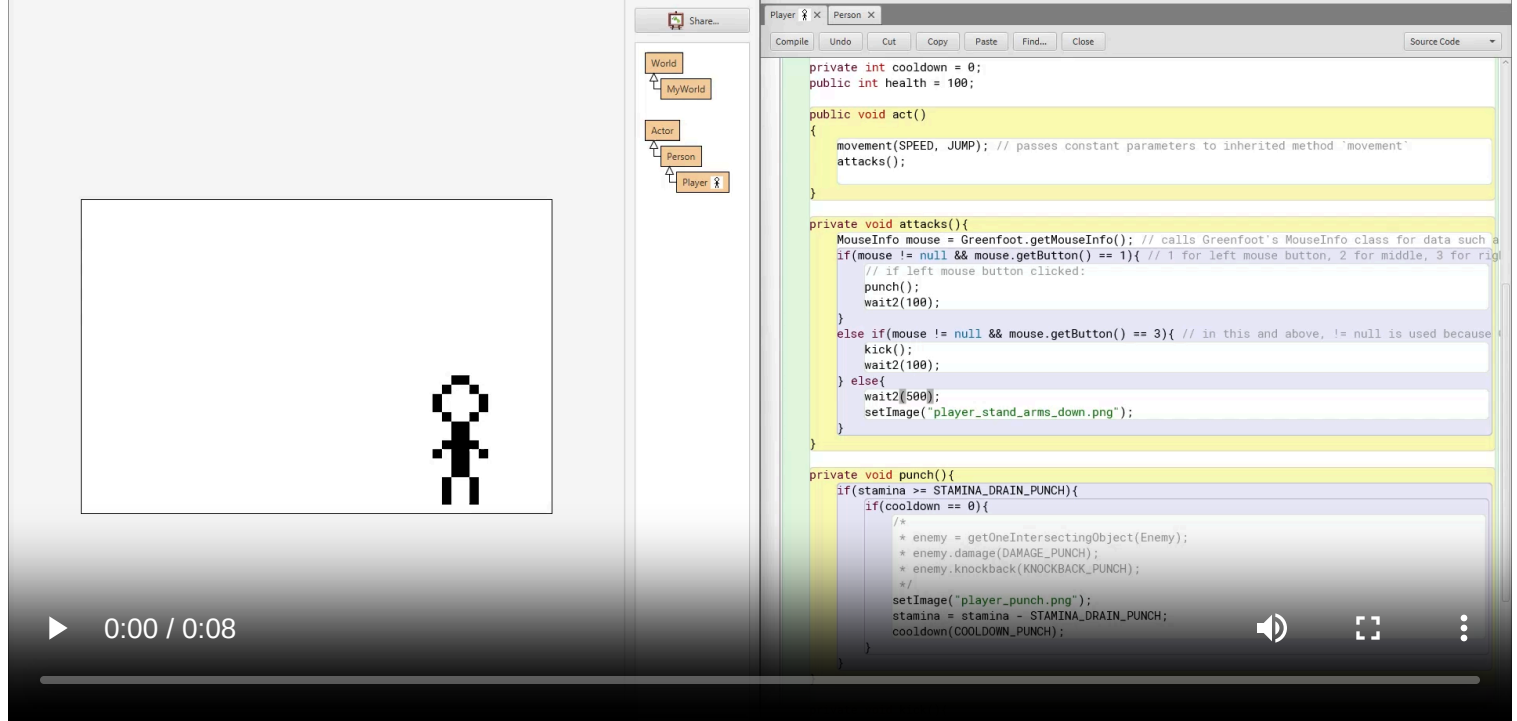
Round() Method

Whilst implementing a `wait2()` method (further explained later), I was going to reuse the rounding code from my `cooldown(double t)` method. Instead of doing this, I optimised the code by introducing a new `round(double target)` function.

```
public int round(double target){  
    DecimalFormat df = new DecimalFormat('###');  
    df.setRoundingMode(RoundingMode.CEILING);  
    return Integer.parseInt(df.format(target));  
}
```

Sprite Setting

Whilst I had technically completed my test plan, I had not accounted for some of the problems which I may have encountered. One of them is that I had not checked that the player's sprite would revert back to the default after an attack sprite is displayed.



(attack_sprite_timing_error.mp4)

```
private void attacks(){
    MouseInfo mouse = Greenfoot.getMouseInfo(); // calls Greenfoot's MouseInfo class for data such as
    if(mouse != null && mouse.getButton() == 1){ // 1 for left mouse button, 2 for middle, 3 for right
        // if left mouse button clicked:
        punch();
        wait2(100);
        setImage("player_stand_arms_down.png");
    }
    if(mouse != null && mouse.getButton() == 3){ // in this and above, != null is used because
        kick();
        wait2(1);
        setImage("player_stand_arms_down.png");
    }
}
```

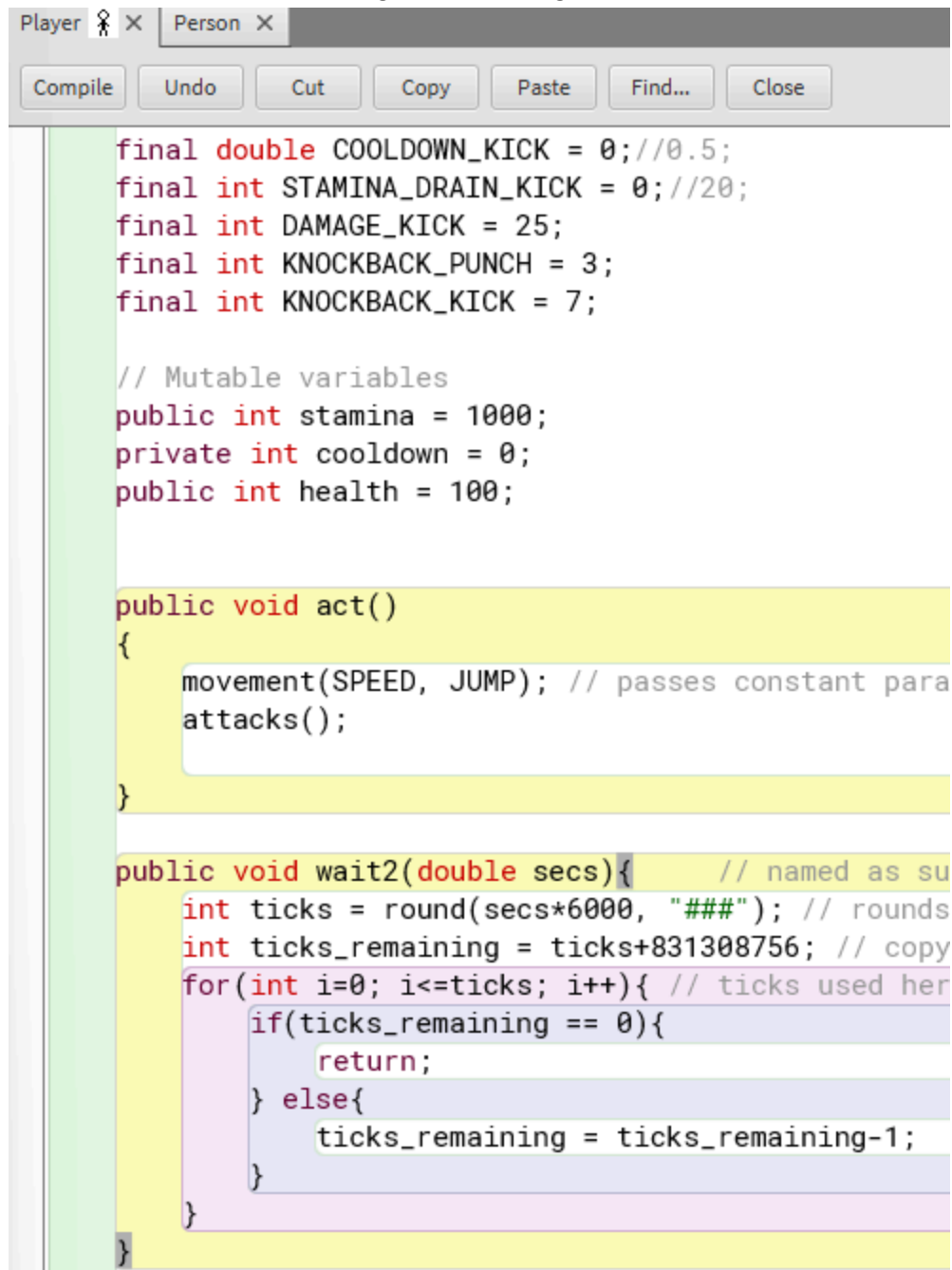
I thought that it would be an easy fix like in the above image. Sadly this does not work. I was able to trace the point of vulnerability to the `wait2()` which I then tried to fix.

```
public void wait2(double secs){ // named as such because wait() is a
    int ticks = round(secs*6000, "###"); // rounds value for how many greenfoot ticks
    int ticks_remaining = ticks+831308756; // copy of ticks to iterate down to 0
    for(int i=0; i<=ticks; i++){ // ticks used here because ticks_remaining is not
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }
}
```

As you can see in

```
int ticks_remaining = ticks+831308756;
```

I tested the function by iterating through arbitrarily large numbers. This is because the core idea of the function is that if each second runs 60 frames, then 60 frames can be populated with a 'spinner loop' to accurately wait 1 second. My test would have accounted for a minimum of 13855145.93333333... seconds. Unfortunately this did not work so I theorised that the error stemmed from the fact that it was an inherited method and so might be running in a different thread.



The screenshot shows a Java IDE window titled 'Player' with a 'Person' tab. The code defines a Player class with several final constants, mutable variables, and two methods: act() and wait2(). The wait2() method contains a loop that decrements ticks_remaining until it reaches zero, effectively waiting for a specified number of seconds.

```
final double COOLDOWN_KICK = 0;//0.5;
final int STAMINA_DRAIN_KICK = 0;//20;
final int DAMAGE_KICK = 25;
final int KNOCKBACK_PUNCH = 3;
final int KNOCKBACK_KICK = 7;

// Mutable variables
public int stamina = 1000;
private int cooldown = 0;
public int health = 100;

public void act()
{
    movement(SPEED, JUMP); // passes constant para
    attacks();
}

public void wait2(double secs){ // named as su
    int ticks = round(secs*6000, "###"); // rounds
    int ticks_remaining = ticks+831308756; // copy
    for(int i=0; i<=ticks; i++){ // ticks used her
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }
}
```

Unfortunately this was to the same effect, even with slight modification as below:

```
public void wait2(int secs){    // named as such because wait() is already
    //int ticks = round(secs*6000, "###"); // rounds value for how many gree
    int ticks_remaining = 831308756; // copy of ticks to iterate down
    /*for(int i=0; i<=secs*60; i++){ // ticks used here because ticks_remai
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }*/
    while(ticks_remaining > 0){
        ticks_remaining--;
    }
}
```

Fixing the Sprite Setting

Today (01.10.26) I fixed the sprite setting issue. I realised that it was caused by the fact that it was the **main act method** which called 60 times per second. This in conjunction with my broken `wait2()` function led to me implementing a manual timer held in the Player act method.

Player  X Person X

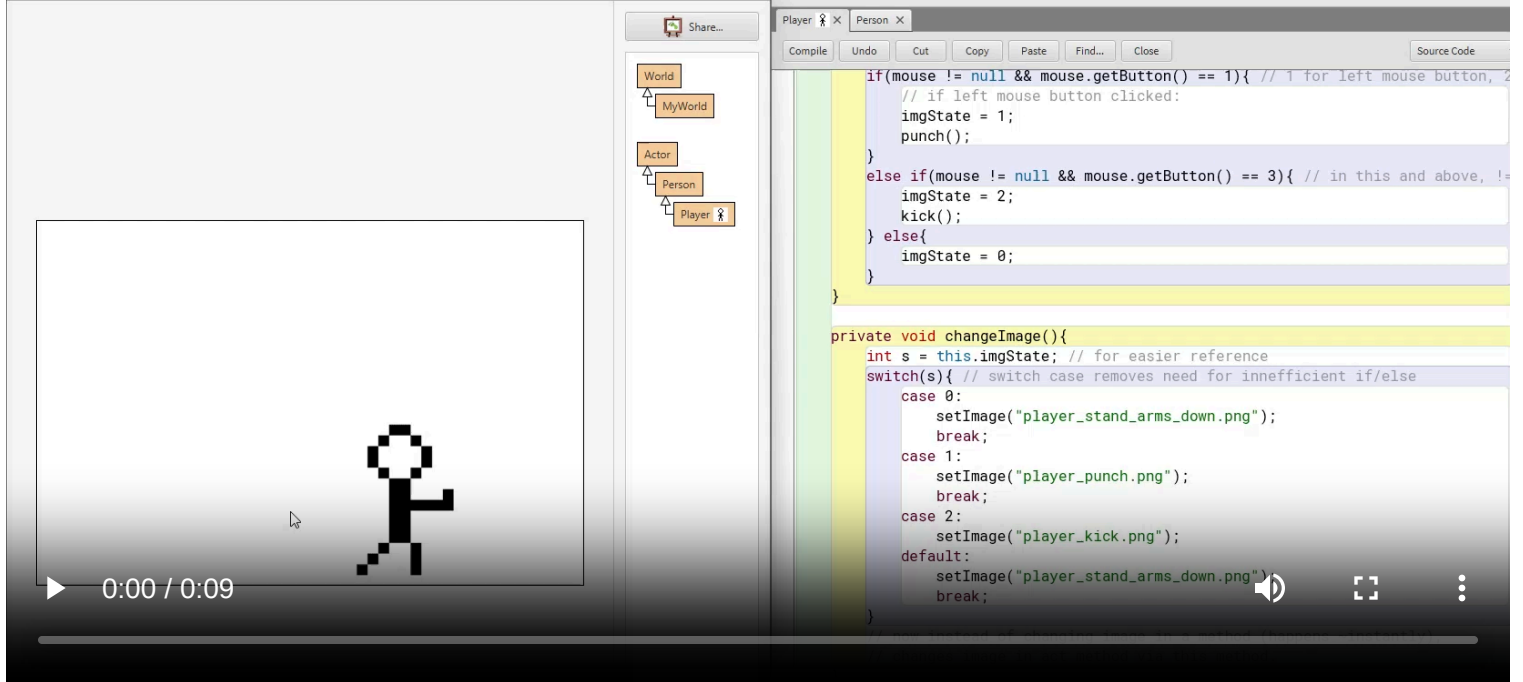
Compile Undo Cut Copy Paste Find... Close Source C

```
final int STAMINA_DRAIN_PUNCH = 10;
final double COOLDOWN_PUNCH = 0.2;
final double COOLDOWN_KICK = 0.5;
final int STAMINA_DRAIN_KICK = 20;
final int DAMAGE_KICK = 25;
final int KNOCKBACK_PUNCH = 3;
final int KNOCKBACK_KICK = 7;

// Mutable variables
public int stamina = 100;
private int cooldown = 0;
public int health = 100;
private int imgState = 0; // 0 as default, 1 for punch, 2 for kick
private int imgCountdown = 0; /* theoretically should limit time for image
change. Each act method this will increment and thus change the image for
sprite.
*/
public void act()
{
    // variables
    imgCountdown++;
    if(cooldown > 0){
        cooldown--;
    }
    if(stamina < STAMINA_MAX){
        stamina++;
    }
    // image changes
    if(imgCountdown == 30){
        changeImage();
        imgCountdown = 0;
    }
}
```

The fact that this solution works is making me consider a solution to the initial wait problem. As a result, in my next stage I might include a similar wait method with the decrementing number is stored in a super class outside of the method. It could be added to with the act method responsible for decreasing the cooldown value once per tick.

Proof:



(fixed_sprite_changes.mp4)